

Utilisation de Git pour Design Assistant

Yuri López de Meneses
27 septembre 2024 v.1.1

1 Introduction

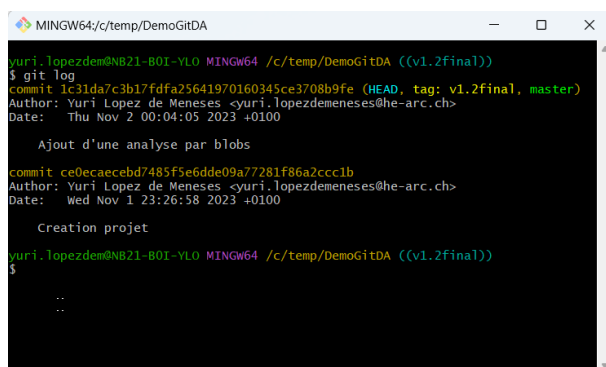
Les projets Matrox Design Assistant sont composés de fichiers majoritairement lisibles, comme XML. Par conséquent la gestion de versions peut être gérée par Git.

Table des matières

1 Introduction	1
2 Installation	1
3 Initialisation d'un <i>repository</i> local	2
4 Stockage d'une nouvelle version	2
5 Voir l'historique	3
6 Voir les modifications	3
7 Revenir à une ancienne version	4
8 Options avancées : remote repository	4
Références	4
A Dépannage	4

2 Installation

Depuis le portail d'entreprise de la HE-Arc on peut installer Git (<https://git-scm.com/downloads>) ou TortoiseGit (<https://tortoisegit.org/>).



```
mingw64/c/temp/DemoGitDA
yuri.lopezdem@NB21-B01-VL0 MINGW64 /c/temp/DemoGitDA ((v1.2final))
$ git log
commit 1c31da7c3b17fdfa25641970160345ce3708b9fe (HEAD, tag: v1.2final, master)
Author: Yuri Lopez de Meneses <yuri.lopezdemeneses@he-arc.ch>
Date: Thu Nov 2 00:04:05 2023 +0100

    Ajust d'une analyse par blobs

commit ce0ecaecbd7485f5e6dde09a77281f86a2ccclb
Author: Yuri Lopez de Meneses <yuri.lopezdemeneses@he-arc.ch>
Date: Wed Nov 1 23:26:58 2023 +0100

    Creation projet

yuri.lopezdem@NB21-B01-VL0 MINGW64 /c/temp/DemoGitDA ((v1.2final))
$
```

FIGURE 1 – Console Git Bash, qui peut être ouverte avec right-click sur le dossier de projet DA.

3 Initialisation d'un *repository* local

Après avoir créé un nouveau projet DA, on ouvre Git Bash dans le répertoire du projet (en faisant right-click sur l'icône du dossier). On initialise le *repository*, on ajoute tous les fichiers et on fait un premier *commit* ou enregistrement.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA
$ git init
Initialized empty Git repository in C:/temp/DemoGitDA/.git/

yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git add .
warning: in the working copy of 'OperatorView/OperatorView.htm', LF will be replaced by CRLF the next time you commit.

yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git commit -m "Creation projet"
[master (root-commit) ce0ecae] Creation projet
46 files changed, 9217 insertions(+)
create mode 100644 ComponentSettingsCache.xml
create mode 100644 DemoGitDA.dap
create mode 100644 DemoGitDA.ui
create mode 100644 MainFlowchart.daf
create mode 100644 MainFlowchart.ui
create mode 100644 OperatorView.dao
[...]
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$
```

4 Stockage d'une nouvelle version

Lorsqu'on a apporté une modification au projet (idéalement une qui fonctionne), on peut enregistrer (commit) une nouvelle version. La commande `git status` listera les fichiers modifiés.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
modified:   DemoGitDA.dap
modified:   MainFlowchart.daf
modified:   MainFlowchart.ui

no changes added to commit (use "git add" and/or "git commit -a")
```

S'il y a des nouveaux fichiers, ce qui arrivera si on a ajouté un *subflowchart* ou *operatorview*, il faudra les ajouter à la liste de fichiers à versionner avec la commande `git add <chemin>`. Pour sélectionner tous les fichiers modifiés ou ajoutés on fait `git add .`

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git add .
```

Ensuite on appelle `git commit` pour enregistrer les modifications en ajoutant une explication pour avoir une meilleure vue historique :

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git commit -m "Ajout d'une analyse par blobs"
[master 1c31da7] Ajout d'une analyse par blobs
3 files changed, 9420 insertions(+), 234 deletions(-)
```

Lors de l'enregistrement, un numéro hexadécimal est affiché (1c31da7 dans l'exemple ci-dessus.) Ce numéro de commit sera utilisé pour s'y référer à cette version, par exemple si on voulait y retourner.

On peut aussi faire ces étapes (`status`, `add`, `commit`) dans une interface graphique qu'on peut lancer avec `git gui &`. Dans cette fenêtre vous pourrez voir les modifications, sélectionner les fichiers à retenir, entrer un message de commit et faire le commit.

Vous pouvez en plus donner un nom particulier ou nom de version, avec la commande `git tag`.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git tag -a v1.2final -m "Rendu final"
```

5 Voir l'historique

Vous pouvez voir l'historique des vos *commits* avec la commande `git log`.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA ((v1.2final))
$ git log
commit 1c31da7c3b17fdfa25641970160345ce3708b9fe (HEAD, tag: v1.2final, master)
Author: Yuri Lopez de Meneses <yuri.lopezdemeneses@he-arc.ch>
Date: Thu Nov 2 00:04:05 2023 +0100
```

Ajout d'une analyse par blobs

```
commit ce0ecaecbd7485f5e6dde09a77281f86a2ccc1b
Author: Yuri Lopez de Meneses <yuri.lopezdemeneses@he-arc.ch>
Date: Wed Nov 1 23:26:58 2023 +0100
```

Creation projet

ou dans sa version compacte (avec `-oneline`), d'une ligne par commit :

```
$ git log --oneline
1c31da7 (HEAD, tag: v1.2final, master) Ajout d'une analyse par blobs
ce0ecae Creation projet
```

6 Voir les modifications

Vous pouvez analyser les modifications ou différences entre l'état actuel de votre projet DA et un commit précédent (typiquement le dernier) grâce à la commande `git diff`, aussi en indiquant spécifiquement pour quel fichier et quel *commit*.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA (master)
$ git diff MainFlowchart.daf
diff --git a/MainFlowchart.daf b/MainFlowchart.daf
index 419ad88..b7b9823 100644
--- a/MainFlowchart.daf
+++ b/MainFlowchart.daf
@@ -1064,7 +1064,7 @@
</subObject>
<input name="NumberOfFerets">
<expression>
-      <expression-string>8</expression-string>
+      <expression-string>16</expression-string>
</expression>
</input>
<input name="PostCalculationSource">
```

Dans l'exemple ci-dessus on regarde les modifications seulement dans le fichier `Mainflowchart.daf` et on peut voir que le paramètre `NumberOfFerets` a été changé de 8 à 16.

7 Revenir à une ancienne version

Si on se rend compte que les modifications ne sont pas bonnes et on veut revenir en arrière, on peut le faire avec `git checkout` et le numéro hexadécimal du commit désiré. Si on veut écraser les modifications actuelles il faut utiliser l'option `-f`.

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp/DemoGitDA ((v1.2final))  
$ git checkout -f 1c31da7  
HEAD is now at 1c31da7 Ajout d'une analyse par blobs
```

8 Options avancées : remote repository

Dans certains cas, typiquement pour un travail en groupe, on peut travailler à partir d'un *repository* commun et qui ne sera donc pas local, mais remote. Si on commence à partir du code déjà existant dans un repository remote, par exemple le gitlab de l'école (<https://gitlab-etu.ing.he-arc.ch> pour les étudiants et <https://labinfo.ing.he-arc.ch/gitlab> pour les personnel de l'Ecole), on le récupère avec la commande `git clone` et le chemin au repository (indiqué par un URL https ou ssh)

```
yuri.lopezdem@NB21-BOI-YLO MINGW64 /c/temp  
$ git clone https://labinfo.ing.he-arc.ch/gitlab/igib/da-templates-examples/moire.git  
Cloning into 'moire'...  
remote: Enumerating objects: 34, done.  
remote: Total 34 (delta 0), reused 0 (delta 0), pack-reused 34  
Receiving objects: 100% (34/34), 62.07 KiB | 12.41 MiB/s, done.  
Resolving deltas: 100% (13/13), done.
```

Si au contraire on commence avec un projet local et on veut lui ajouter un repository remote, on l'ajoutera dans la configuration du projet avec

```
git remote add origin https://labinfo.ing.he-arc.ch/gitlab/igib/da-templates-examples/moire.git
```

où `origin` est le nom par défaut par lequel on s'y référera au repository remote.

Qu'on ait obtenu le repository par clone ou en l'ajoutant à posteriori, ensuite on procédera pareil : Avant de faire un commit local on actualisera notre repository local à partir du remote avec

```
git pull origin main
```

et après avoir fait le commit local de nos modifications, on les reportera vers le remote avec

```
git push origin main
```

Dans les 2 exemples `main` est la "branche" principale. Si vous travaillez avec plusieurs branches, vous pouvez indiquer laquelle. Pour plus d'information sur les branches, s'y référer au manuel git [1].

Ainsi la séquence typique sera

```
git pull origin main  
...  
git commit -m "nouvelle modification"  
...  
git push origin main
```

Références

- [1] Scott CHACON et Ben STRAUB. *Pro Git*. 2024. URL : <https://git-scm.com/book/en/v2>.

A Dépannage

Pour chercher de l'aide, faire `git <commande> --help`, ce qui ouvrira une page web d'aide.